Sardar Patel Institute of Technology

Department of Computer Engineering

Academic Year 2026-27

Repo-Up: Repository Intelligence System for Code Analysis, Auto-Repair and Sandbox Verification

K V S Varshith, Dipesh Shitkar, Aneesh, B.Tech.(Comp.)

Project Guide(s) : Dr. Rupali Sawant

Abstract

Repo-Up is a web tool that inspects an uploaded codebase for quality, security and maintainability. Tree-sitter ASTs for five languages feed a dependency graph, CK-style metrics and rule-based checks, producing an ISO/IEC 25010-aligned readiness score. An LLM-assisted repair engine proposes patches, which an isolated sandbox validates before acceptance. On a synthetic benchmark all 25 files parsed successfully, with recall of 0.50–0.75.

Introduction

AI-assisted ("vibe") coding yields working code fast, but often without clear structure, tests or safe secret handling. Students, indie developers and teams reviewing AI-generated code need a tool that inspects the real codebase and explains what is wrong.

Objective(s)

- Analyse Python, JavaScript, TypeScript, Java and C++ code.
- Quantify quality with coupling and cohesion metrics.
- Detect structural and security defects with explainable rules.
- Verify proposed fixes in a sandbox first.

Problem Definition

Given a codebase of unknown quality, detect its structural, security and maintainability defects, score it on a 0–100 scale aligned to ISO/IEC 25010, and offer only fixes shown not to break the code.

Contribution

- **Unified pipeline:** AST → graph → metrics → rules, one scoring rubric.
- **Tiered repair engine:** deterministic fixes and LLM suggestions behind a 5-step patch validator.
- **Sandbox with rollback:** patches run in an ephemeral copy; failures are reverted.
- **Live dashboard and benchmark:** SSE streaming UI and a reproducible P/R/F1 harness.

Design

React 19 + TypeScript front-end with a FastAPI back-end (Fig. 1).

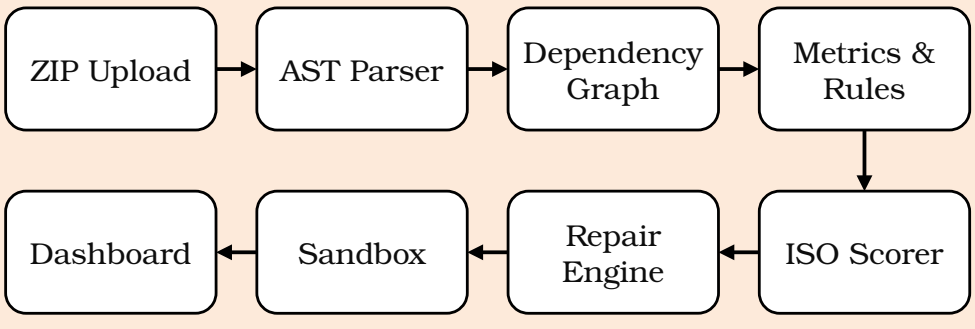


Fig. 1: Repo-Up processing pipeline

Methodology/Algorithms

- **Ingestion:** ZIP upload with zip-slip, symlink and size-limit checks.
- **Parsing and graph:** Tree-sitter ASTs yield a directed NetworkX import graph; cycles are found with Tarjan's SCC [3].
- **Metrics:** COF = E / (n^2 – n), afferent coupling, WMC, LCOM and DIT proxies, rated against thresholds [1].
- **Rules:** 11 structural and security checks, e.g. missing tests, committed .env, hard-coded secrets, eval/exec.
- **Scoring:** sub-scores map to Modularity, Analysability, Modifiability and Testability of ISO/IEC 25010 [2]:

$$\text{Score} = 0.5 \cdot \text{Metrics} + 0.3 \cdot \text{Structural} + 0.2 \cdot \text{Security}$$

- **Repair:** Tier 1 deterministic patches (yaml.safe_load, weak hashes); Tier 2/3 LLM guidance. Each patch passes path, line-bound, exact-match, non-empty and parseability checks.
- **Sandbox:** an ephemeral copy is checked by AST parsing, static and security regression checks and unit tests; any failure triggers rollback.

Results

Synthetic benchmark: 5 repositories per language, 25 files.

Language	TP	FP	FN	Prec.	Recall	F1
Python	2	29	2	0.06	0.50	0.11
JavaScript	3	27	1	0.10	0.75	0.18
TypeScript	3	27	1	0.10	0.75	0.18
Java	3	29	1	0.09	0.75	0.17
C++	3	27	1	0.10	0.75	0.18

- **Parsing:** 25/25 files parsed; 0 parser or analysis failures.
- **Detection:** recall 0.50–0.75, but precision only 0.06–0.10 as generic rules over-report on small fixtures.
- **Security:** weak-crypto rule scored P = R = 1.00 in JS, TS, Java and C++.

Conclusion

A single AST-based pipeline can analyse five languages, explain findings through ISO/IEC 25010 and verify fixes in a sandbox before they reach the user. Parsing was fully reliable, but low precision shows the rules need tuning. Future work: rule refinement and evaluation on real-world repositories.

References

[1] K. A. M. Ferreira et al., "Identifying thresholds for object-oriented software metrics," J. Syst. Softw., 85(2), 2012.

[2] ISO/IEC 25010:2011, System and software quality models.

[3] R. Tarjan, "Depth-first search and linear graph algorithms," SIAM J. Comput., 1(2), 1972.